PEB

PROGRAMACION ESTRUCTURADA BASICA

UNIDAD 2

Matrices

Material preparado por: Dr. Ing. Pablo Martín Vera

INDICE

Contenido

1. MATRICES 2
1.1 DECLARACIÓN 4
1.2 INICIALIZACIÓN 5
2. MATRICES COMO PARÁMETROS DE FUNCIONES 5
3. MANIPULACIÓN DE MATRICES 6
3.1 RECORRIDO POR FILAS 6
3.2 RECORRIDO POR COLUMNAS 7
3.3 ACCESO DIRECTO 7
4. ALGORITMOS 8
4.1 SUMA POR FILAS 9
4.2 SUMA POR COLUMNAS 9
4.3 MÁXIMOS Y MÍNIMOS MÚLTIPLES 10

OBJETIVOS: *trabajar con colecciones de datos de dos dimensiones para poder trabajar con programas que necesiten organizar la información en base a dos parámetros.*

1. Matrices

Las matrices son colecciones de datos todos del mismo tipo y en memoria continua al igual que los vectores. Pero la diferencia radica en la forma de acceso a cada posición de memoria y en cómo se declararán.

Las matrices son arrays de dos dimensiones mientras que los vectores tienen una sola dimensión. El lenguaje C admite arrays de más dimensiones, pero en esta materia solo usaremos hasta 2.

Una matriz se declara especificando la cantidad de elementos que contiene, pero separada en dos dimensiones. Por ejemplo

```
int mat[4][5];
```

Declara una matriz con 4 elementos en su primera dimensión y 5 en su segunda. La interpretación que se le da a este tipo de estructuras es similar a una tabla donde la primera dimensión indica la cantidad de filas y la segunda la cantidad de columnas. También puede interpretarse con un vector donde cada posición a su vez contiene otro vector.

Entonces la declaración anterior define en memoria una matriz de 4 filas y 5 columnas de la siguiente forma:

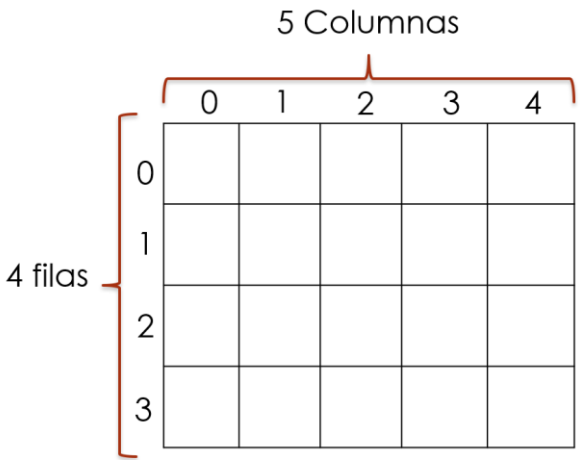

Figura 1: representación de una matriz

Donde las filas al ser 4 van de 0 a 3 y las columnas al ser 5 van de 0 a 4. Es decir que la colección es de 20 valores enteros que sale de la multiplicación de la cantidad de filas por la cantidad de columnas.

En Memoria los datos se guardan en forma consecutiva por fila, es decir primero el dato de la fila 0 columna 0, luego de la fila 0 columna 1, fila 0 columna 2 y así sucesivamente.

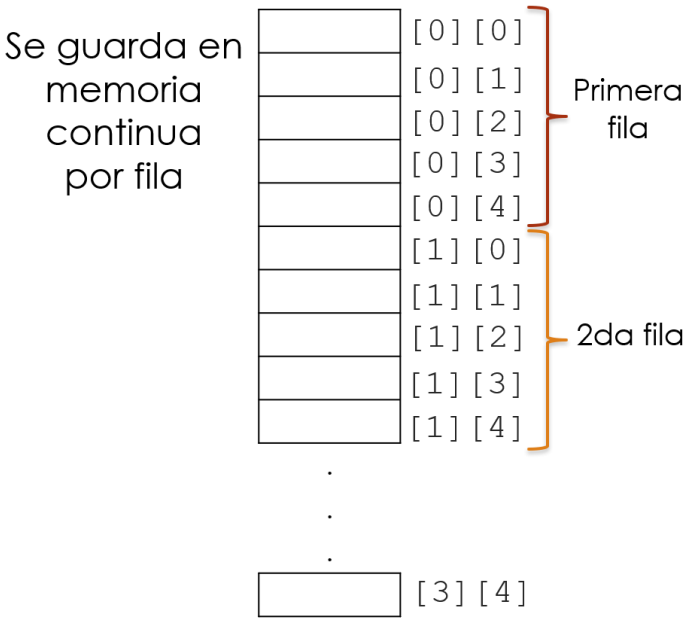

Figura 2: forma en que la matriz se almacena en memoria

Para acceder a cada variable de la colección se utilizan ahora 2 subíndices, uno para identificar la fila y otro para la columna. El primer subíndice hace referencia a la fila y el segundo a la columna.

En la siguiente figura se muestra como se accede a la posición marcada utilizando dos subíndices

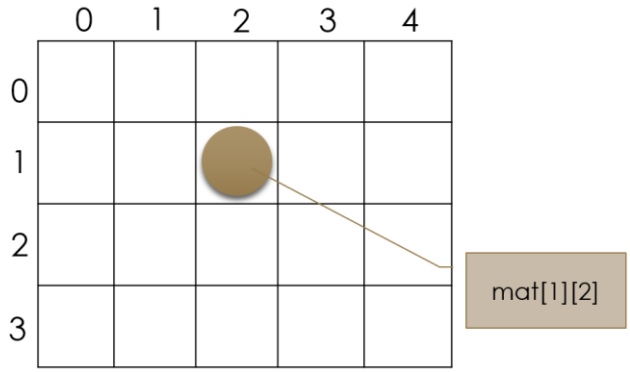

Figura 3: forma de acceso a una posición de la matriz

Internamente se realiza un cálculo para llegar a la posición de memoria donde se encuentra el dato. Recordemos que todos los datos se encuentran uno a continuación del otro y los subíndices son un desplazamiento desde la dirección de inicio. En este caso como hay dos subíndices el desplazamiento se calcula con ambos. El primer subíndice, el de las filas, indica la cantidad de posiciones que debe desplazarse hasta llegar donde comienza la información de dicha fila. Pero al desplazarse una fila en realidad debe desplazarse tantas direcciones de memoria como columnas tenga la matriz. Por lo tanto, el primer subíndice se multiplica internamente por la cantidad de columnas y luego a ese valor se le suma el segundo subíndice para poder realmente calcular la cantidad de direcciones que se debe desplazar desde el inicio. Entonces en el ejemplo se desplazará 7 posiciones desde el inicio ((1*5) + 2). Puede verse que para poder posicionarse en la memoria conocer el dato de la cantidad de columnas es fundamental.

***Importante**: las matrices son colecciones de datos todos del mismo tipo. Es una forma para acceder a los datos considerando dos parámetros para posicionarse, por ejemplo, detalle de ventas por mes de cada vendedor. Bajo ningún punto de vista la matriz debe contener datos mezclados de distintas entidades, aunque el tipo de dato coincida. Por ejemplo, usar la columna 0 para guardar un código y en las restantes cantidades. Si bien el lenguaje lo permite no es una buena práctica y no debe utilizarse de ese modo. En ese caso lo que se puede hacer es crear un vector paralelo a las filas con los códigos para separar conceptualmente los datos y que no estén mezclados en la misma colección*

1.1 Declaración

Formato:

```
tipo_dato  nombre [cantidad de filas][cantidad de columnas];
```

Ejemplo:

```
float tabla[20][10];
```

De esta forma, se reservan 200 posiciones en memoria, (producto de 20 filas por 10 columnas), donde se podrán almacenar valores float. Recordemos que una matriz es un array al igual que un vector entonces en la variable lo que se almacena es la dirección de memoria de donde se encuentra el primer dato. Es un puntero estático ya que la dirección de memoria no puede ser modificada.

Recordemos además que los arrays son colecciones estáticas y por lo tanto una vez definido su tamaño no puede modificarse. Por lo tanto debe dimensionarse según las necesidades del problema a resolver teniendo en cuenta la cantidad máxima de elementos que se podrían almacenar

1.2 Inicialización

También es factible inicializar una matriz.

Ejemplo 1:

```
int matriz [2][3]={1,2,3,4,5,6};
```

Los valores son asignados por filas. Primero se asignan los elementos de la primera fila, luego los elementos de la segunda fila, y así sucesivamente. En el ejemplo la matriz quedaría con los siguientes valores:

```
matriz[0][0]=1 matriz[0][1]=2    matriz[0][2]=3
matriz[1][0]=4 matriz[1][1]=5    matriz[1][2]=6
```

Ejemplo 2:

```
int matriz [][3]={1,2,3,4,5,6};
```

En este ejemplo no se especifican la cantidad de filas. Las mismas se calculan en forma automática según la cantidad de elementos con las que inicializa la matriz. El parámetro de la cantidad de columnas NUNCA puede dejarse vacío ya que el compilador no tendría forma de calcular las direcciones de memoria para acceder a cada dato.

Ejemplo 3:

```
int matriz [2][3] ={
                    {1,2,3},
                    {4,5,6}
                    };
```

Otra nomenclatura posible para inicializar una matriz. Esta forma es visualmente más adecuada que las anteriores ya que referencia al formato de una tabla y se puede visualizar rápidamente la ubicación de cada uno de los valores.

Muchas veces será necesario inicializar la matriz en 0 si es que se desea utilizarla para contar o acumular. Para esos casos y solo al momento de declarar la variable es posible inicializarla en 0 de la siguiente forma:

```
int matriz [2][3] = {{0}};
```

Esta nomenclatura solo es válida para el 0 y al momento de declarar la variable.

2. Matrices como parámetros de funciones

Como se explicó anteriormente la variable del tipo array (ya sea un vector o una matriz) contienen la dirección de memoria del primer dato, por lo tanto al enviar como parámetro a una función un array, los cambios que se hagan dentro de la función se verán reflejados desde donde se la llamó ya que se trabaja con la misma dirección de memoria.

Al enviar un vector se podía dejar sin completar el tamaño en la especificación de los parámetros de la función ya que al ser una dirección de memoria y el lenguaje no valida límites ese dato es irrelevante. En cambio, al enviar una matriz al tener más de una dimensión SIEMPRE se debe indicar la cantidad de columnas, pudiendo dejar vacío solo la cantidad de filas. Esto se debe a que el compilador necesita saber cómo están organizados los datos para hacer los cálculos de a qué dirección de memoria dirigirse cuando se escriben los subíndices de la matriz. Indicándole la

cantidad de columnas y debido a que los datos se guardan en forma consecutiva, puede calcular cuantas direcciones de memoria debe desplazarse hasta llegar a la fila indicada.

3. Manipulación de Matrices

El orden más natural de procesar los vectores es el orden secuencial: del primero al último elemento. En el caso de los arreglos bidimensionales, existen diferentes órdenes para su recorrido. Los más usuales son: recorrido por filas y recorrido por columnas. En el primero, se recorre la primera fila (desde la primera columna hasta la última), luego la segunda fila, etc.; En el segundo, se recorre la primer columna (desde la primer fila hasta la última), luego la segunda columna, etc.

Por supuesto, también se puede acceder directamente a un elemento (característica de todos los arreglos).

3.1 Recorrido por filas

Se utiliza un subíndice para acceder a cada fila. Con el valor fijo de cada fila, se recorre con el otro subíndice todas las columnas de dicha fila.

El siguiente código muestra una función para el ingreso por teclado de los valores de una matriz cargando los datos por fila. Es decir, se cargan primero todos los datos de la fila 0, luego los de la fila 1 y así hasta finalizar. El dato de la cantidad de columnas podría omitirse ya que la función esta armada para cargar una matriz entera de 5 columnas (sin importar la cantidad de filas). Debido a que la cantidad de columnas es un dato requerido al especificar una matriz como parámetro, las funciones ya no se pueden hacer tan genéricas como sí es posible hacerlas para los vectores. Es decir que esta función servirá para ingresar datos en una matriz entera, sin importar la cantidad de filas, pero que tenga 5 columnas.

```
void CargaMatrizPorFilas(int mat[][5], int cf, int cc)
{
        int f,c;
        for (f=0;f<cf;f++)
        {
                for (c=0;c<cc;c++)
                {
                    printf("Ingrese un numero para fila %d columna %d: ", f,c);
                    scanf("%d",&mat[f][c]);
                }
        }
}
```

El recorrido por filas se utiliza también para visualizar una matriz en forma de tabla en pantalla, para ello se imprime toda la primera fila, un salto de página, toda la segunda fila y así sucesivamente.

La siguiente función permite mostrar una matriz de enteros en forma de tabla, recibe la matriz, la cantidad de filas y la cantidad de columnas.

Para lograr la visualización en formato tabla, cada elemento de la fila se separa con ancho de campo o con una tabulación y al finalizar de mostrar todos los datos de la fila se agrega un salto de línea para que los datos de la siguiente fila se muestren debajo encolumnados con los anteriores.

En la codificación para mostrar datos encolumnados no es recomendable usar tabulaciones ya que si la matriz tiene muchas columnas no entraría en pantalla. Es mejor encolumnar los datos utilizando ancho de campo en el formato del printf dependiendo del largo de los datos que se almacenan. Por ejemplo, si la matriz que se quiere mostrar tiene números enteros de 2 cifras como máximo en el printf usaremos el formato %3d para indicarle que cada valor lo muestre en 3

lugares, de forma que quede un espacio delante y luego el dato. El código de la función es el siguiente:

```
void mostrarMatriz(int mat[][5], int cf, int cc)
{
      int f,c;
      for (f=0;f<cf;f++)
      {
            for (c=0;c<cc;c++)
               printf("%3d",mat[f][c]);
         printf ("\n");
      }
}
```

3.2 Recorrido por columnas

Se invierte el orden de los ciclos for, el exterior va a ser el que recorre las columnas y el interior el que recorre las filas. Se utiliza un subíndice para acceder a cada columna. Con el valor fijo de cada columna, se recorre con el otro subíndice todas las filas de dicha columna.

```
void CargaMatrizPorColumnas(int mat[][5], int cf, int cc)
{
      int f,c;
      for (c=0;c<cc;c++)
      {
            for (f=0;f<cf;f++)
            {
                  printf("Ingrese un numero para columna %d fila %d:
               ", c,f);
                  scanf("%d",&mat[f][c]);
            }
      }
}
```

3.3 Acceso directo

Al igual que en los vectores es posible acceder directamente a cualquier posición de una matriz indicando subíndices válidos para las filas y columnas sin necesidad de cargar la matriz entera. En estos casos si se le solicita al usuario el número de fila y de columna donde desea carga el dato, el número ingresado DEBE validarse para asegurarse de que corresponde con una fila o columna válida.

A modo de ejemplo se realiza el siguiente programa: Se desea registrar la cantidad de ventas realizadas en el año en una empresa. Para ello se ingresan los datos de las facturas indicando por cada una:

- el mes (1 a 12)
- número de sucursal (entero de 1 a 5)
- cantidad de ventas (entero)

La carga de las facturas finaliza con una cantidad menor o igual a 0. Al finalizar mostrar los datos en forma de tabla.

```
#include <stdio.h>
void MostrarVentas(int [][12], int, int);
int main()
{
    int ventas[5][12]={{0}}, cant, mes, suc;
    printf("Ingrese la cantidad vendida: ");
    scanf("%d",&cant);
    while (cant>0)
    {
        do
        {
            printf("Ingrese el mes:");
            scanf("%d",&mes);
        }while (mes<1||mes>12);



        do
        {
            printf("Ingrese la sucursal:");
            scanf("%d",&suc);
        }while (suc<1||suc>5);
        ventas[suc-1][mes-1]+=cant;
        printf("Ingrese la cantidad vendida: ");
        scanf("%d",&cant);
    }
    MostrarVentas(ventas,5,12);
    return 0;
}

void MostrarVentas(int v[][12], int cf, int cc)
{
    int i,f,c;
    printf ("Sucursal/Mes");

    for (i=1;i<=12;i++)
        printf ("%5d", i);

    for (f=0;f<cf;f++)
    {
        printf ("\n%12d",f+1);
        for (c=0;c<cc;c++)
            printf ("%5d", v[f][c]);
    }
}
```

También es habitual tener un vector en paralelo a la matriz. Si en el ejemplo anterior los códigos de sucursales no fueran números correlativos, entonces habría un vector paralelo con códigos (que se cargarán con el algoritmo de carga sin duplicados) y luego para poder posicionarse en la fila de la matriz relacionada con determinado código primero ser hará una búsqueda sobre el vector y luego se accede en forma paralela en la matriz según el índice del vector donde se encontró el código deseado.

4. Algoritmos

Sobre las matrices pueden aplicarse distintos tipos de algoritmos que variaran según el problema a resolver. A continuación, se muestran los más comunes.

4.1 Suma por filas

Muchas veces es necesario realizar una suma de todos los valores cada una de las filas de una matriz. Para ello se recorre la matriz por filas y suman todos los valores de la misma. En el ejemplo anterior sumar por fila nos permitirá conocer la cantidad total de unidades vendidas por cada sucursal. El resultado serán tantas sumas como filas tenga nuestra matriz, por lo tanto, para guardar esos valores se genera un vector en paralelo a las filas de la matriz. La figura 4 muestra un ejemplo de un vector que guarda los resultados de la suma de cada una de las filas de una matriz de enteros.

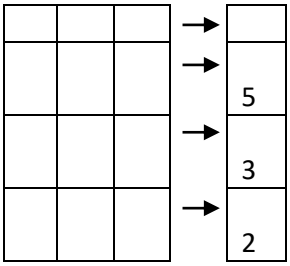

Figura 4: vector resultante de la suma por filas de los valores de una matriz

A continuación, el código de la función para sumar las filas de una matriz entera de 5 columnas. En este caso se utiliza una variable suma para ir acumulando, pero esta variable puede obviarse y sumar directamente en el vector resultado, previamente poniendo en 0 dicha posición o iniciar el vector previamente todo en 0.

```
void SumaPorFilas(int mat[][5], int vec[], int cf, int cc)
{
    int f,c,suma;
    for (f=0;f<cf;f++)
    {
        suma=0;
        for (c=0;c<cc;c++)
            suma += mat[f][c];
        vec[f]=suma;
    }
}
```

4.2 Suma por columnas

Del mismo modo es posible realizar la suma de todos los valores de cada una de las columnas de la matriz. Para ello se recorre la matriz por columna sumando todos sus valores. Aplicándolo al ejemplo anterior obtendríamos un vector con la información total de unidades vendidas en cada uno de los meses del año ya que se estaría sumando la cantidad vendida en cada sucursal para cada mes. La figura 5 muestra un ejemplo de un vector que guarda los resultados de la suma de cada una de las columnas de una matriz de enteros.

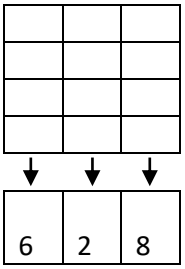

Figura 5: vector resultante de la suma por columnas de los valores de una matriz

A continuación, se muestra el código de una función que permite sumar por columna una matriz de enteros. En este ejemplo directamente se va acumulando sobre el vector sin usar una variable intermedia para calcular la suma.

```
void SumaPorColumnas(int mat[][5], int vec[], int cf, int cc)
{
    int f,c,suma;
    for (c=0;c<cc;c++)
    {
        vec[c]=0;
        for (f=0;f<cf;f++)
            vec[c] += mat[f][c];
    }
}
```

4.3 Máximos y Mínimos múltiples

De la misma forma que con los vectores es posible obtener el valor máximo o mínimo de una matriz y detectar las repeticiones. Recordando que este tipo de algoritmos tiene dos etapas, la primera calcular el máximo valor o el mínimo y luego volver a recorrer para indicar las posiciones donde se encuentra.

Se toma como referencia el primer valor (posición 0,0) y luego se recorre toda la matriz con un doble for comparando con dicho valor de referencia. Una vez obtenido el valor deseado se vuelve a recorrer la matriz para ver donde se repite dicho valor.

```
#include <stdio.h>
void CargaMatriz(int[][5], int, int);
int ValorMinimo(int[][5], int, int);
void MostrarValor(int[][5],int,int,int);
void MostrarMatriz(int [][5], int, int);

int main()
{
    int mat[3][5], min;
    CargaMatriz(mat,3,5);
    min = ValorMinimo(mat,3,5);
    MostrarMatriz(mat,3,5);
    MostrarValor(mat,min,3,5);
    return 0;
}
```

```
void CargaMatriz(int m[][5], int cf, int cc)
{
    int f,c;
    for (f=0;f<cf;f++)
        for (c=0;c<cc;c++)
        {
            printf("Ingrese un numero \
                (en fila %d columna %d):", f+1,c+1);
            scanf("%d",&m[f][c]);
        }
}

void MostrarMatriz(int m[][5], int cf, int cc)
{
    int f,c;
    for (f=0;f<cf;f++)
    {
        for (c=0;c<cc;c++)
            printf("%4d", m[f][c]);
        printf("\n");
    }
}

int ValorMinimo(int m[][5], int cf, int cc)
{
    int min = m[0][0], f,c;
    for (f=0;f<cf;f++)
        for (c=0;c<cc;c++)
            if (m[f][c]<min)
                min = m[f][c];
    return min;
}
```

```
void MostrarValor(int m[][5],int valor, int cf, int cc)
{
    int cont=0,i;
    printf("El valor es: %d y se encuentra en las \
            siguientes posiciones de la matriz:\n",valor);

    int min = m[0][0], f,c;
    for (f=0;f<cf;f++)
        for (c=0;c<cc;c++)
            if (m[f][c]==valor)
            {
                printf("fila %d columna %d\n", f+1,c+1);
                cont++;
            }

    printf ("El valor se repite %d veces", cont);
}
```

En el ejemplo a las posiciones que se muestran se les suma 1 para que sea más amigable y natural para el usuario.